Software Requirements Specification (SRS)

VaxTrace

A Smart Vaccination System for Parents

Final Year Design Project

Prepared by: Ashis Biswas

Student ID: _____

Supervisor: _____

Institution: Daffodil International University

Document Version 1.0

June 2026

Document Control

This Software Requirements Specification (SRS) is a controlled document. The revision history below records each version and the substantive changes applied.

Table 1: Document Revision History

Version	Date	Author	Summary of Changes
1.0	June 2026	Ashis Biswas	Initial release of the VaxTrace Software Requirements Specification, prepared in accordance with IEEE Std 830-1998. Covers the introduction and objectives, overall description, system requirements, functional requirements with a use-case diagram, non-functional requirements (ISO/IEC 25010), system architecture, data model (ERD), system workflow, expected outcomes, future enhancements, and IEEE references.

Table of Contents

Document Control....2
Table of Contents....3
List of Figures....5
List of Tables....5
1. Introduction....6
1.1 Purpose....6
1.2 Objective....6
1.3 Scope....6
1.4 Definitions, Acronyms, and Abbreviations....7
2. Overall Description....7
2.1 Product Perspective....7
2.2 User Classes and Characteristics....8
2.3 Operating Environment....8
2.4 Design and Implementation Constraints....8
2.5 Assumptions and Dependencies....8
3. System Requirements....9
3.1 Hardware Requirements....9
3.2 Software Requirements....9
4. Functional Requirements....10
4.1 Use-Case Diagram....10
4.2 Representative Use-Case Description....12
4.3 Functional Requirements List....12
5. Non-Functional Requirements....13
6. System Architecture....14
6.1 Architecture Overview....14
6.2 Component Description....16
7. Data Model....16
7.1 Entity-Relationship Diagram....16
7.2 Collection Descriptions....18
8. System Workflow....18
9. Expected Outcomes....20
10. Future Enhancements....20

11. Conclusion 20
12. References 20

List of Figures

Figure *1*: VaxTrace Use-Case Diagram 11
Figure *2*: VaxTrace System Architecture 15
Figure *3*: VaxTrace Entity-Relationship (Data Model) Diagram 17
Figure *4*: VaxTrace Vaccination Workflow 19

List of Tables

Table 1: Document Revision History 2
Table *2*: Definitions, Acronyms, and Abbreviations 7
Table *3*: Hardware Requirements 9
Table *4*: Software Requirements 9
Table *5*: Functional Requirements 12
Table *6*: Non-Functional Requirements (ISO/IEC 25010) 13
Table *7*: MongoDB Collection Descriptions 18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of VaxTrace, a smart, web-based vaccination management system for parents, healthcare workers, and administrators. The document is prepared in accordance with the IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998) [8] and serves as the authoritative reference for the design, implementation, and evaluation of the system as a Final Year Design Project. The intended audience comprises the project supervisor and examination committee, the development team, and future maintainers of the system.

1.2 Objective

The objective of VaxTrace is to design and develop a centralized, web-based vaccination management system that digitizes child immunization records for parents, healthcare workers, and administrators. The system automatically generates a complete vaccination schedule for each registered child based on the date of birth, in accordance with Bangladesh's National Immunization Schedule (NIS) [7], and proactively notifies parents through SMS and email before each scheduled dose. By replacing manual, paper-based immunization tracking with an automated digital workflow, VaxTrace aims to:

- **O1 —** Reduce the number of missed or delayed vaccine doses among children.
- **O2 —** Give parents real-time, self-service visibility into their child's immunization history and upcoming doses.
- **O3 —** Give healthcare workers a fast, reliable interface for registering children and recording administered vaccines.
- **O4 —** Give administrators centralized oversight of vaccine inventory, coverage rates, and missed-dose statistics.
- **O5 —** Demonstrate, as a Final Year Design Project, a practical full-stack implementation of an automated, rule-driven scheduling and notification system.

1.3 Scope

VaxTrace is a scalable, web-based platform for centralized vaccination record management, built as a single full-stack application using the Next.js App Router with JavaScript for both the user interface and the backend API [1], with MongoDB as the persistent data store [2]. Three role-based user classes — parents/guardians, healthcare workers, and administrators — access the system through dedicated dashboards served by the same application.

In scope:

- Child registration and duplicate-registration prevention.
- Automatic generation of the NIS-based vaccination schedule from the date of birth.
- Recording of administered doses and automatic calculation of the next due date.
- Multi-channel reminder notifications (SMS and email) driven by scheduled jobs.

- Vaccine inventory monitoring and administrative reporting and statistics.
- Role-based authentication and access control.

Out of scope (addressed as future enhancements in Section 10):

- Integration with national Birth Registration / EPI databases.
- A dedicated native mobile application.
- Full bilingual (Bangla/English) interface and notifications.
- AI-driven coverage-gap and outbreak-risk analytics, and telemedicine modules.

The current scope is limited to a single-country deployment aligned with Bangladesh's NIS [7] and is intended primarily as a working prototype for academic evaluation.

1.4 Definitions, Acronyms, and Abbreviations

Table 2: Definitions, Acronyms, and Abbreviations

Term	Definition
NIS	National Immunization Schedule (Bangladesh).
EPI	Expanded Programme on Immunization.
API	Application Programming Interface.
JWT	JSON Web Token.
RBAC	Role-Based Access Control.
NextAuth	Authentication library for Next.js used for session management.
Cron Job	A time-based scheduled task that runs automatically at fixed intervals.
SMTP	Simple Mail Transfer Protocol.
Atlas	MongoDB Atlas, the managed cloud database service for MongoDB.
DOB	Date of Birth.
UI	User Interface.
ERD	Entity-Relationship Diagram.
SRS	Software Requirements Specification.
NFR	Non-Functional Requirement.
CRUD	Create, Read, Update, Delete.

2. Overall Description

2.1 Product Perspective

VaxTrace is a new, self-contained product rather than an extension of an existing system. It is built as a single Next.js application in which frontend pages and backend API routes are served from the same codebase [1]. At a high level, the system consists of role-based web

interfaces for parents, healthcare workers, and administrators; Next.js API routes (/api/*) that handle business logic and data access; NextAuth for session management [3] combined with a JWT [4] and RBAC middleware layer for role-based access control; scheduled cron jobs that check vaccination due dates and trigger reminders; a notification layer that dispatches reminders through an email service (Nodemailer) [5] and an SMS gateway API; and a centralized MongoDB Atlas database [9].

2.2 User Classes and Characteristics

- **Parent / Guardian (Web User):** Views their child's vaccination history and upcoming schedule, and receives SMS and email reminders ahead of due dates. Assumed to have basic web/smartphone literacy.
- **Healthcare Worker (Web User):** Registers children, records administered vaccines, and updates vaccination status. Assumed to have routine clinical data-entry experience.
- **Administrator (Web User):** Manages user accounts and permissions, monitors vaccine inventory, and generates reports and statistical analyses. Assumed to be the most technically proficient user class.

2.3 Operating Environment

- **Client:** Modern web browsers (Chrome, Firefox, Edge, Safari) on desktop or mobile devices.
- **Application server:** Node.js runtime running Next.js (App Router, JavaScript) for both frontend and backend.
- **Database:** MongoDB Atlas (cloud) or a self-hosted MongoDB instance [9].
- **Notification gateways:** Third-party SMS gateway API and SMTP-based email service.
- **Deployment:** Cloud hosting (e.g., Vercel) or a local server environment with HTTPS enabled.

2.4 Design and Implementation Constraints

- Each child's vaccination schedule must be automatically generated from the date of birth, following Bangladesh's National Immunization Schedule [7].
- Sensitive child and guardian data must be protected through proper authentication and role-based access control.
- SMS and email reminder delivery depends on third-party gateway availability and may be simulated within the prototype.
- The system must be implemented as a single Next.js (App Router, JavaScript) application with MongoDB, per the project specification.
- The system must operate within the academic project (Final Year Design Project) timeline and resource constraints.

2.5 Assumptions and Dependencies

- Users have access to an internet-connected device with a modern web browser.

- Guardian contact details (mobile number and/or email) are accurate and kept up to date for reminder delivery.
- A third-party SMS gateway and an SMTP email service are available; in the prototype these may be simulated.
- The official Bangladesh NIS dose schedule is available and used as the rule base for automatic schedule generation [7].
- The hosting platform supports scheduled (cron) execution for the reminder jobs.
- MongoDB Atlas (or an equivalent MongoDB deployment) is available as the persistent data store [9].

3. System Requirements

This section defines the minimum hardware and software requirements needed to develop, deploy, and run VaxTrace.

3.1 Hardware Requirements

Table 3: Hardware Requirements

Component	Minimum Requirement	Recommended
Application server (cloud or local)	1 vCPU, 2 GB RAM, 10 GB storage	2 vCPU, 4 GB RAM, SSD storage
Database server (MongoDB Atlas or self-hosted)	Shared cluster (M0) or 2 GB RAM	Dedicated cluster, 4 GB+ RAM
Developer workstation	Dual-core CPU, 8 GB RAM	Quad-core CPU, 16 GB RAM, SSD
Client device	Smartphone, tablet, or PC with a modern browser	Smartphone or laptop with stable internet
Network connectivity	Stable internet connection	Broadband / 4G or better

3.2 Software Requirements

Table 4: Software Requirements

Layer	Technology / Tool	Purpose
Frontend & Backend	Next.js (App Router, JavaScript)	Unified full-stack framework serving UI pages and /api/* routes
Database	MongoDB / MongoDB Atlas	Persistent storage for users, children, vaccines, records, schedules, notifications, inventory
Authentication	NextAuth, JWT, RBAC middleware	Session management and role-based access control
Notifications	Nodemailer (SMTP), SMS	Automated email and SMS reminders

Layer	Technology / Tool	Purpose
	Gateway API	for upcoming doses
Scheduling	Cron jobs (Node.js scheduler / Vercel Cron)	Periodic checks for upcoming vaccination due dates
Hosting / Deployment	Vercel or Node.js server with HTTPS	Production deployment of the application
Version Control	Git and GitHub	Source code management and collaboration
Development Environment	Visual Studio Code, Node.js, npm	Local development and dependency management
Password Security	bcrypt	Hashing of user passwords before storage

4. Functional Requirements

This section presents the system's functional behaviour. The use-case view (Section 4.1) identifies the actors and the services they invoke; Section 4.2 details a representative use case; and Section 4.3 enumerates the testable functional requirements with priority and objective traceability.

4.1 Use-Case Diagram

Figure 1 shows the three actors — Parent/Guardian, Healthcare Worker, and Administrator — and the use cases each can perform within the VaxTrace system boundary.

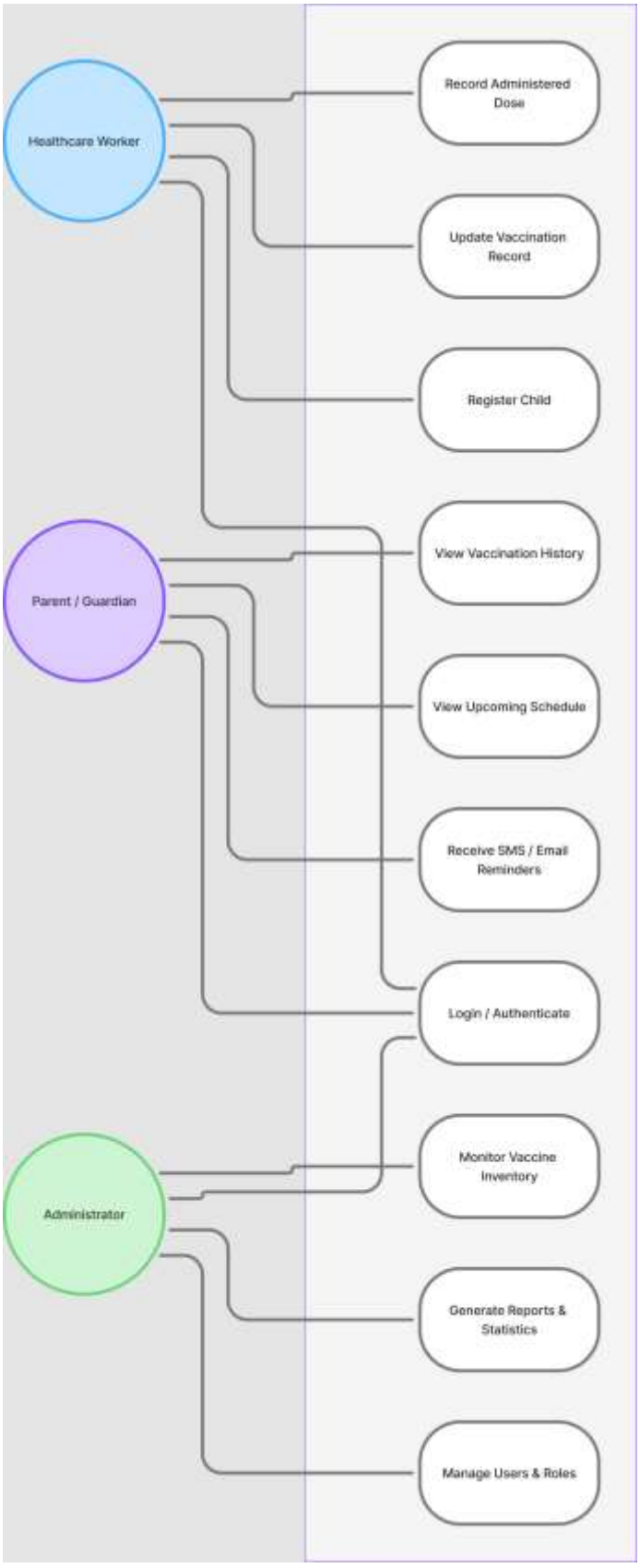


Figure 1: VaxTrace Use-Case Diagram

4.2 Representative Use-Case Description

To illustrate the dynamic behaviour of the system, the core automated use case — sending a vaccination reminder — is described below.

Field	Description
Use Case	UC-08: Send Vaccination Reminder
Primary Actors	System (Cron Job); Parent/Guardian (recipient)
Precondition	The child has one or more scheduled doses and a valid guardian contact on file.
Trigger	The scheduled cron job runs and finds a dose due within the configured reminder window.
Main Flow	1) Cron job scans the schedules collection. 2) It identifies doses due within the window. 3) The notification service composes the message. 4) It dispatches the reminder via the SMS gateway and email (Nodemailer). 5) The dispatch outcome is logged in the notifications collection.
Postcondition	The reminder is recorded and the parent/guardian is notified ahead of the due date.
Exceptions	If a gateway is unavailable, delivery is retried up to three times and then marked as failed.

4.3 Functional Requirements List

Priority levels: H = High (core), M = Medium, L = Low. The Objective column traces each requirement to the objectives defined in Section 1.2.

Table 5: Functional Requirements

ID	Requirement	Pri.	Obj.
FR-1	The system shall authenticate users using NextAuth with JWT-based sessions, enforced by an RBAC middleware layer for parents, healthcare workers, and administrators.	H	O2–O4
FR-2	Healthcare workers shall register a new child with details such as name, date of birth, gender, and guardian contact information.	H	O3
FR-3	The system shall prevent duplicate child registration based on identifying details such as guardian contact and date of birth.	M	O3
FR-4	The system shall automatically generate a complete vaccination schedule for each child based on date of birth, following Bangladesh's NIS [7].	H	O1, O5
FR-5	Healthcare workers shall update vaccination records after a dose is administered, including date administered, vaccine name, and dose number.	H	O3
FR-6	The system shall automatically calculate and update the next due date immediately after a vaccination record is updated.	H	O1, O5

ID	Requirement	Pri.	Obj.
FR-7	Parents/guardians shall access a dashboard to view their child's complete vaccination history and upcoming scheduled doses.	H	O2
FR-8	A scheduled cron job shall check upcoming vaccination due dates and automatically trigger SMS and email reminders to parents before each scheduled date.	H	O1, O5
FR-9	Administrators shall manage user accounts and role-based permissions for healthcare workers and parents.	M	O4
FR-10	Administrators shall monitor vaccine inventory, including stock levels and batch/expiry tracking.	M	O4
FR-11	The system shall generate reports and statistical analyses, including immunization coverage rates and missed-dose tracking, for administrators.	M	O4
FR-12	The system shall maintain audit logs of all vaccination record updates.	L	O4

5. Non-Functional Requirements

The non-functional requirements are organised according to the ISO/IEC 25010 software product quality model [10], each with a measurable acceptance criterion.

Table 6: Non-Functional Requirements (ISO/IEC 25010)

ID	Quality Attribute	Requirement and Acceptance Criterion
NFR-1	Security	Passwords are stored only as bcrypt hashes (cost factor ≥ 10); all traffic is served over HTTPS (TLS 1.2+); access is enforced per role via JWT and RBAC; controls align with the OWASP Top Ten [11].
NFR-2	Performance Efficiency	Under normal load (up to ~100 concurrent users), 95% of API requests complete in under 3 seconds.
NFR-3	Reliability	Target availability of 99% (prototype); automated daily MongoDB backups; failed SMS/email deliveries are retried up to 3 times with structured error handling.
NFR-4	Maintainability	Code is organised into feature modules with documented API routes and is managed under Git version control.
NFR-5	Scalability	The architecture supports a growing number of children, guardians, and facilities through stateless API routes and horizontally scalable hosting and MongoDB Atlas.
NFR-6	Usability	Role-specific dashboards are usable by non-technical parents; primary tasks are completable within three clicks; the UI is responsive on mobile and desktop.
NFR-7	Portability	The application runs on any modern browser and is deployable to Vercel or any Node.js host with HTTPS.

ID	Quality Attribute	Requirement and Acceptance Criterion
NFR-8	Compatibility	The system interoperates with third-party SMS gateways and SMTP email services through standard APIs.

6. System Architecture

6.1 Architecture Overview

VaxTrace is implemented as a single Next.js application using the App Router for both the user interface and the backend API routes [1]. Parents, healthcare workers, and administrators all interact with the same Next.js frontend, which communicates with the backend through RESTful API routes under /api/*. The backend is supported by three core components: NextAuth for session management [3], a JWT [4] and RBAC middleware layer that enforces role-based access control on every request, and scheduled cron jobs that check for upcoming vaccination due dates. When a reminder is due, the cron job triggers the notification layer, which dispatches messages through an email service (Nodemailer) [5] and an SMS gateway API. All application data — users, children, vaccines, vaccination records, schedules, notifications, and inventories — is persisted in MongoDB Atlas [9]. Figure 2 illustrates this architecture.

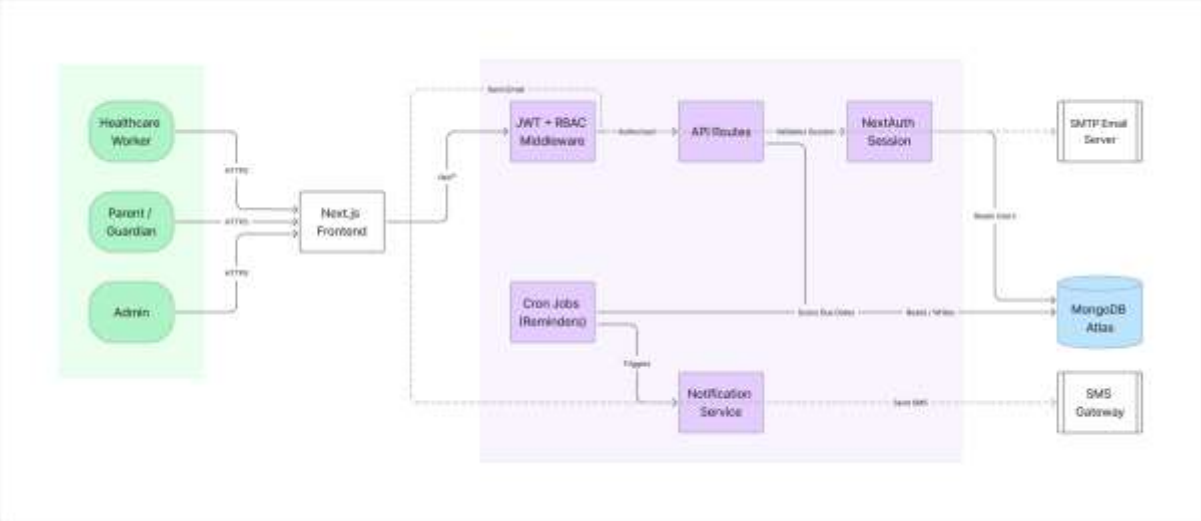


Figure 2: VaxTrace System Architecture

6.2 Component Description

- **Next.js Frontend:** Renders role-specific dashboards and forms for parents, healthcare workers, and administrators using the App Router.
- **Next.js Backend (API Routes):** Exposes RESTful endpoints under /api/* that implement business logic, validation, and data access.
- **NextAuth Session:** Manages login sessions and issues JWTs on successful authentication.
- **JWT + RBAC Middleware:** Validates tokens and enforces role-based permissions on every protected request.
- **Cron Jobs (Reminders):** Run on a schedule to identify vaccinations due within a configurable window and queue reminder notifications.
- **Notification Layer:** Dispatches due reminders through Nodemailer (email) and a third-party SMS gateway API.
- **MongoDB Atlas:** Centralized data store holding the users, children, vaccines, vaccinationRecords, schedules, notifications, and inventories collections.

7. Data Model

7.1 Entity-Relationship Diagram

Figure 3 presents the VaxTrace data model as an entity-relationship diagram. It defines the seven MongoDB collections, their key fields, and the relationships and cardinalities between them — for example, one guardian (user) may have many children, and each child has one or more scheduled doses.

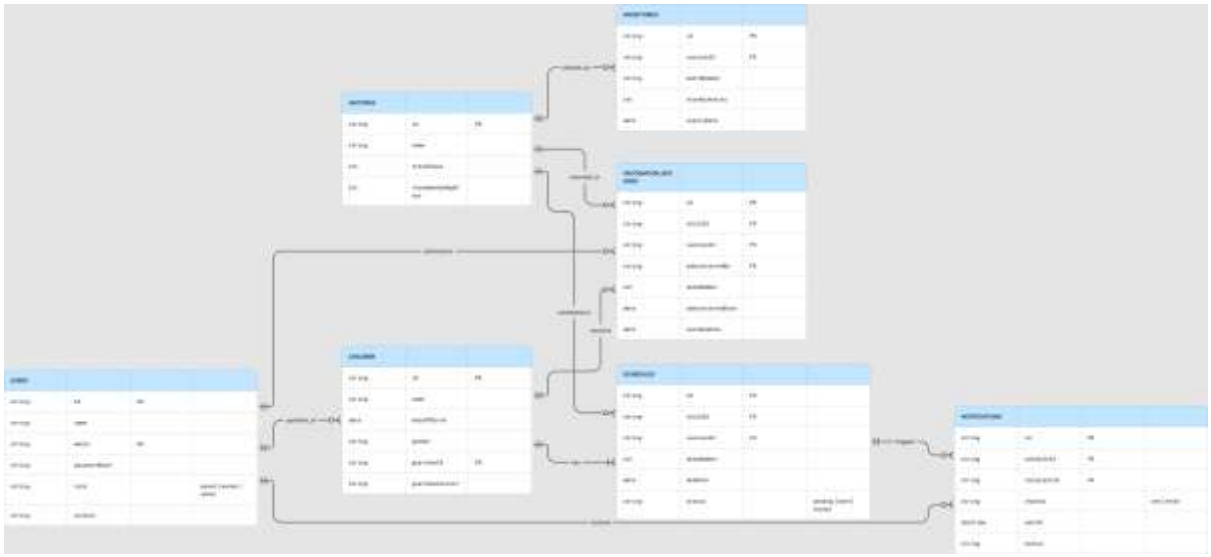


Figure 3: VaxTrace Entity-Relationship (Data Model) Diagram

7.2 Collection Descriptions

Table 7: MongoDB Collection Descriptions

Collection	Purpose	Key Fields
users	Accounts for all roles (parent, worker, admin).	id, name, email, passwordHash, role, contact
children	Registered children and guardian linkage.	id, name, dateOfBirth, gender, guardianId, guardianContact
vaccines	Vaccine catalogue aligned with the NIS.	id, name, totalDoses, recommendedAgeDays
schedules	Generated per-child dose schedule.	id, childId, vaccineId, doseNumber, dueDate, status
vaccinationRecords	Administered-dose records.	id, childId, vaccineId, administeredBy, doseNumber, administeredDate, nextDueDate
notifications	Reminder dispatch log.	id, scheduleId, recipientId, channel, sentAt, status
inventories	Vaccine stock and batch tracking.	id, vaccineId, batchNumber, stockQuantity, expiryDate

8. System Workflow

Figure 4 illustrates the end-to-end workflow of VaxTrace: from user login and role-based branching, through child registration, automatic schedule generation, persistence, the cron job's due-date checks and reminder delivery, dose administration and record update, and finally report generation.

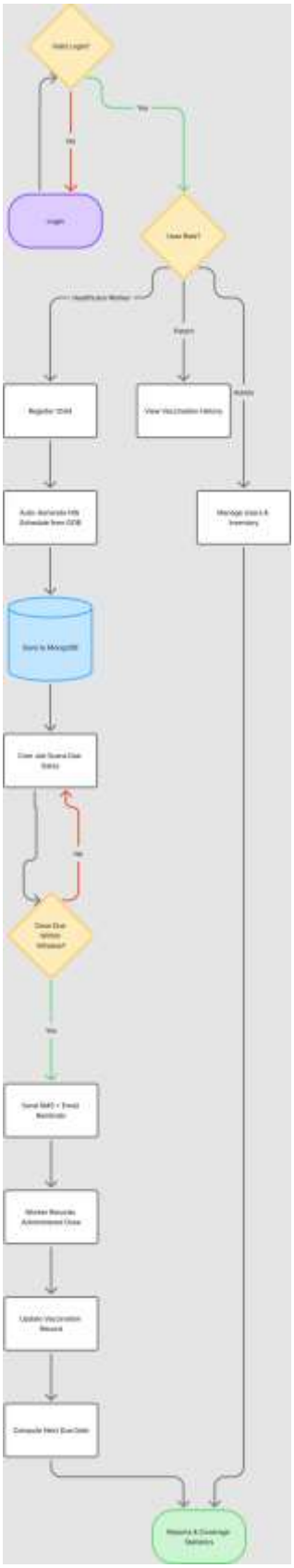


Figure 4: VaxTrace Vaccination Workflow

9. Expected Outcomes

Upon completion, VaxTrace is expected to deliver the following outcomes:

- A fully functional, role-based web application covering child registration, automated NIS-based scheduling, vaccination record updates, and reminder notifications.
- A measurable reduction in missed or delayed vaccine doses through proactive, automated SMS and email reminders sent ahead of each due date.
- A centralized, digital alternative to paper-based immunization records that improves data accuracy, auditability, and accessibility for parents and healthcare workers.
- A self-service parent dashboard that increases visibility into each child's vaccination history and upcoming schedule without requiring a clinic visit.
- An administrative reporting capability that surfaces immunization coverage rates and missed-dose statistics to support data-driven decision-making.
- Real-time vaccine inventory monitoring, including stock levels and batch/expiry tracking, to reduce stock-outs and wastage at healthcare facilities.
- A secure system — with hashed passwords, JWT-based authentication, role-based access control, and HTTPS — that protects sensitive child and guardian data.
- A working prototype suitable for demonstration and evaluation as a Final Year Design Project, with an architecture extensible toward the enhancements in Section 10.

10. Future Enhancements

- Integration with the national Birth Registration / EPI database for automated child verification.
- A dedicated mobile application for parents (Android/iOS).
- Full bilingual support (Bangla and English) across the interface and notifications.
- Vaccine coverage-gap and outbreak-risk analysis using AI-driven analytics.
- A telemedicine / teleconsultation module for post-vaccination queries.
- Integration with national health information systems to enable nationwide deployment.

11. Conclusion

VaxTrace provides a centralized, easy-to-use web platform for managing child immunization records in line with Bangladesh's National Immunization Schedule [7]. By combining a single Next.js application with automated scheduling and multi-channel reminders backed by MongoDB, the project demonstrates a practical approach to reducing missed vaccinations and improving record-keeping for parents, healthcare workers, and administrators alike. The requirements, architecture, data model, and workflow defined in this SRS provide a complete and verifiable basis for the implementation and defense of the system.

12. References

[1] Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs

[2] MongoDB, Inc., "MongoDB Documentation." [Online]. Available: https://www.mongodb.com/docs/

[3] "NextAuth.js Documentation." [Online]. Available: https://next-auth.js.org/

[4] "Introduction to JSON Web Tokens (JWT)." [Online]. Available: https://jwt.io/introduction

[5] "Nodemailer Documentation." [Online]. Available: https://nodemailer.com/about/

[6] World Health Organization, "Expanded Programme on Immunization (EPI)." [Online]. Available: https://www.who.int/teams/immunization-vaccines-and-biologicals

[7] Directorate General of Health Services (DGHS), Bangladesh, "National Immunization Schedule, EPI Bangladesh."

[8] IEEE, IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications," Institute of Electrical and Electronics Engineers, 1998.

[9] MongoDB, Inc., "MongoDB Atlas Documentation." [Online]. Available: https://www.mongodb.com/docs/atlas/

[10] ISO/IEC 25010:2011, "Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models," International Organization for Standardization, 2011.

[11] OWASP Foundation, "OWASP Top Ten Web Application Security Risks." [Online]. Available: https://owasp.org/www-project-top-ten/